

Security essentials

Fix four small mistakes that attackers love: redirects, secret checks, logs and SQL.

The big idea

A backend is open to the whole internet, so some requests come from **attackers**. Most attacks use small, ordinary mistakes, not clever magic.

Think of the locks on a house: none is complicated, but every door needs one. The rule behind all four: **never trust what comes in from outside**.

New words

redirect

A response that says "go to this other address instead".

?next=

Where to send the user after login: `/login?next=/tasks/`.

host

The website part of an address, like `backendlab.test`.

API key

A long secret password that a program sends to prove who it is.

personal data

Anything that identifies a real person, like an email.

parameterised query

SQL with a `%s` placeholder; the value is passed **separately**.

Attack 1: the open redirect

An attacker emails Ada a link:

`backendlab.test/login?next=https://backendlab-login.evil.example`. It starts with your real site, so she logs in. Your site then sends her to the attacker's copy, which asks for her password again.

- The link looked safe because it started with your address
- **Fix:** only redirect to your own site; anything else goes to /
- Your function: `safe_next_url(url, allowed_hosts)`

Attack 2: the timing attack

A guard checks a password one letter at a time and says "wrong!" at the first mismatch. A wrong first letter gets a quick answer; a right one, a slightly slower one. By timing answers, an attacker guesses the secret **one letter at a time**.

- Python's `==` on strings can behave like that guard
- **Fix:** `hmac.compare_digest` always takes the same time
- Your function: `api_key_matches(provided, expected)`

Attack 3: personal data in logs

Your app logs "Password reset for ada.lovelace@example.com". Logs get copied into tools, emailed around and kept for months. One day a log file leaks, and thousands of emails are public.

- In many countries that is also against the law
- **Fix:** mask it first: `a***@example.com` is still useful for debugging
- Your function: `mask_email(email)`

Attack 4: SQL injection

A search box glues the typed title into SQL. Someone types `x' OR '1'='1`. The quote ends the title early, and the rest becomes **part of the command**: "title is x, OR 1 equals 1". Every task of every user comes back.

- With other text, an attacker could even delete tables
- **Fix:** pass the value separately, so it's treated as plain data
- Your function: `find_tasks_by_title(title)`

What your code receives and returns

The tests call your four functions with these values. `HOSTS` is `["backendlab.test"]`.

```
safe_next_url("/dashboard/", HOSTS)           # "/dashboard/"
safe_next_url("https://evil.example/login", HOSTS) # "/"
safe_next_url("//evil.example", HOSTS)        # "/"

api_key_matches("sk_live_abc123", "sk_live_abc123") # True
api_key_matches("", "")                             # False: empty never matches

mask_email("ada.lovelace@example.com")          # "a***@example.com"
mask_email("not-an-email")                      # "***"

find_tasks_by_title("Deploy")                   # [1], the Deploy task's id
find_tasks_by_title("x' OR '1'='1")            # [] (nothing has that title)
```

Tool: `url_has_allowed_host_and_scheme`

Django's own "safe to redirect to?" **function**: address first, then the allowed hosts.

```
from django.utils.http import url_has_allowed_host_and_scheme
urls = ["/cart/", "https://shop.test/cart/",
        "https://evil.example/", "//evil.example"]
for url in urls:
    print(url, url_has_allowed_host_and_scheme(url, allowed_hosts=["shop.test"]))
```

OUTPUT

```
/cart/ True
https://shop.test/cart/ True
https://evil.example/ False
//evil.example False
```

Tools: not value and hmac.compare_digest

`not` catches empty and `None` at once; `compare_digest` compares **bytes** in constant time.

```
import hmac

print(not "", not None, not "abc")
print(hmac.compare_digest("tea".encode(), "tea".encode()))
print(hmac.compare_digest("tea".encode(), "tee".encode()))
```

OUTPUT

```
True True False
True
False
```

Tools: `partition("@")` and `text[0]`

`partition` always gives 3 pieces; a missing `@` leaves the last two empty.

```
print("tea@shop.test".partition("@"))
print("no-at-sign".partition("@"))
print("@shop.test".partition("@"))
print("tea"[0], repr(None or ""))    # handy trick: None or "" gives ""
```

OUTPUT

```
('tea', '@', 'shop.test')
('no-at-sign', '', '')
('', '@', 'shop.test')
t ''
```

Gluing SQL vs a parameterised query

`f"... title = '{title}'"`

- The value is glued **into the SQL text**
- A quote in it ends the string early
- `x' OR '1'='1` becomes part of the command
- `Ada's task` crashes the query

`cursor.execute(sql, [values])`

- Write a bare `%s` in the SQL, **no quotes**
- Pass the values in a **separate list**
- `"... WHERE id = %s", [pen.id]`
- Quotes in the value are just letters

Your task, step by step

- 1 `safe_next_url`: not empty and safe? Return `url`; otherwise `"/`" → tests 1-2
- 2 `api_key_matches`: if not provided or not expected: return `False` → test 4
- 3 Then `hmac.compare_digest(...)` on both keys, `.encode()`d → tests 3, 5
- 4 `mask_email`: `local, at, domain = (email or "").partition("@");`
any empty? `***"` → test 7
- 5 Otherwise first letter of `local` + `***@` + domain, with an f-string → test 6
- 6 `find_tasks_by_title`: change only `cursor.execute`: bare `%s`, `[title]` → tests 8-10

Common mistakes

- `url.startswith("/")` as the check: `//evil.example` sneaks through
- Returning `True/False` from `safe_next_url`: it must return an **address**
- `provided == expected`: not constant time, and `"" == ""` is `True`
- `email.split("@")` crashes on `None`; `partition` always gives 3 pieces
- `'%s'` with quotes, or an f-string: the value must go in `[title]`

Where you'll use this

Django already protects you in many places: its login view checks `?next=` with the same function you used, and the ORM always uses parameterised queries. The danger is where you step **outside** that protection.

- Raw SQL for a report: always pass values in the list
- Checking a webhook's or partner's API key: `compare_digest`
- Writing log lines about users: mask emails, never log passwords

Check yourself

1. Why is a link that starts with your real site still dangerous?
2. What does `==` leak about a secret that `compare_digest` doesn't?
3. Why should `mask_email("not-an-email")` return `"***"`?
4. In `x' OR '1'='1`, which character lets the text escape into the SQL?

Keep learning

[SQL Injection Tutorial – What is SQL Injection and How to Prevent it](#)

freeCodeCamp · article · beginner. Start here: shows how string-built SQL gets hijacked and why parameterized queries stop it.

[Security in Django](#)

Django docs · docs · beginner. A tour of what Django protects you from automatically and where you still have to be careful.

[Unvalidated Redirects and Forwards Cheat Sheet](#)

OWASP · article · intermediate. Explains the open-redirect attack your `safe_next_url` function blocks with `url_has_allowed_host_and_scheme`.

[hmac.compare_digest](#)

Python docs · docs · intermediate. Reference for the constant-time comparison you use for API keys, so timing does not leak the secret.

[Performing raw SQL queries: passing parameters](#)

Django docs · docs · intermediate. Shows the safe way to pass values to `cursor.execute` with placeholders instead of string formatting.